Student Skill Exchange

Concise Pre-Defense Assessment Report

Field	Value
Diploma topic	Development of a Peer-to-Peer Skill Exchange Platform for University Students
Team	Makhmutov Nursultan; Burlibaev Tamir (SE-2413)
Supervisor	Konstantin Latuta
Institution	Astana IT University
Prepared	24 October 2026
Repository	https://github.com/HashiraOfSun/student-skill-exchange
Submission version	pre-defense-1

This concise report consolidates the eleven required assessment areas. It preserves the decisions, evidence, traceability, diagrams, and limitations needed for review while referring readers to the repository for full research records, requirement catalogues, test cases, references, and risk details. The current deliverable is documentation plus an interactive prototype; no production application or completed implementation test results are claimed.

Contents and evidence status

1. Project proposal and scope source

2. Relevance evidence and student research source

3. Literature review source

4. Existing systems comparison source

5. Technology comparison and selected stack source

6. Requirements source

7. System architecture source

8. Interactive UI prototype source

9. Verification and evaluation plan source

10. Project management, responsibilities, and risks source

11. Supervisor feedback and revisions source

Checkpoint status

Item	Status
SSE-1 through SSE-11 documentation	Reviewed and merged into main
SSE-12 assessment package	PR #16 reviewed and merged
Repository version	Tag and release pre-defense-1 created
University form	Administrative submission/confirmation must be retained separately
Runnable application	Not implemented at this checkpoint
Verification results	Tests are planned; no execution results claimed

1. Project proposal and scope

Source document: docs/project-proposal.md

Purpose and problem

Student Skill Exchange is a proposed web platform for university students who want to teach practical skills and learn from peers. The starting problem is that information about students' capabilities and learning needs is scattered across general chats and personal contacts. This makes it difficult to identify a person whose needs are complementary: student A can teach something student B wants, while student B can teach something student A wants.

The project aim is to develop and evaluate a web platform that helps students find compatible partners and arrange peer-to-peer exchanges. The work covers research, requirements, design, an interactive prototype, implementation planning, verification planning, and later development of a connected application.

Target users and main scenario

The initial users are university students, with early research focused on Astana IT University. The first version supports one-to-one exchanges.

1. A student creates an account and signs in.
2. The student records skills they can offer and skills they want to learn.
3. The student searches manually or views reciprocal recommendations.
4. The student opens a profile and sends an exchange request.
5. The recipient accepts or declines the request.
6. If accepted, both participants record the topic, time, and meeting format.
7. The learning session takes place in person or through an external service.

MVP scope

Included	Excluded
Registration, login, and access control	Payments and paid tutoring
Profiles with offered and wanted skills	Built-in chat or video calls
Skill/category search	Native mobile applications
Reciprocal partner suggestions with explanations	Machine-learning recommendations
Incoming/outgoing requests and status history	Certificates or formal skill assessment
Agreed topic, time, and meeting format	Group classes and course marketplace

Included	Excluded
Responsive web interface	Hosting-provider-specific features

Objectives and expected outcomes

- Investigate current help-seeking behaviour and student expectations.
- Review research and comparable systems.
- Define functional and non-functional requirements with verifiable criteria.
- Compare technology options and establish an initial architecture.
- Build and integrate accounts, profiles, skills, discovery, matching, requests, and agreements.
- Test the main journey and compare reciprocal matching with a manual-search baseline.
- Maintain API, database, setup, testing, and user documentation.

The expected final outcome is a working web application and an evidence-based evaluation. The current checkpoint is earlier: it delivers reviewed analysis, design, prototype, planning, and traceability. A production application and measured implementation results are not claimed in this report.

Responsibilities

Member	Primary responsibility
Nursultan Makhmutov	Backend, database, REST API, authentication, authorization, validation, matching, request processing, backend tests, API documentation
Tamir Burlibaev	User flows, interface design, frontend, API integration, responsive behaviour, accessibility, loading/empty/error states
Both	Research, requirements, technology decisions, integration, evaluation, reporting, and peer review

2. Relevance evidence and student research

Source document: docs/research/user-interviews.md

Method

Six Russian-language online questionnaire responses collected from 10 to 23 October 2026 were analysed. Four respondents reported bachelor's study and two reported master's study. All six records, including neutral and internally inconsistent answers, were retained. The study is exploratory: recruitment channel, response rate, university affiliation, field of study, and individual consent selections were not available in the extract. Results must not be generalized to all AITU students.

The questionnaire replaced the discussion format in the original task. It measured recent need for learning help, search route, outcome, difficulty, desired and offered skills, participation preference, meeting format, time budget, desired MVP features, trust signals, and hypothetical use intention.

Descriptive findings

Measure	Result (n=6)
Needed help	3-5 times: 2; 6+ times: 1; none: 2; do not remember: 1
First search route	Known student: 3; paid tutor/mentor: 1; other: 1; no search: 1
Outcome	Suitable help: 3; person found but no session: 1; help did not solve issue: 1; no search: 1
Main difficulty	Finding a suitable person: 2; assessing knowledge: 2; discomfort asking: 1; no search: 1
Participation	Reciprocal exchange: 2; receive help only: 2; not ready to participate: 2
Format	Flexible online/in-person: 3; in person only: 2; online only: 1
Use intention	Definitely: 2; probably: 3; unsure: 1

Five respondents selected search by skill. Three selected offered/wanted skill profiles, and three selected the combined proficiency/language filter option. Built-in chat, meeting-time choice, request controls, and notifications were each selected once. Trust preferences varied: reviews, university confirmation, examples of work, and recommendation from a known person were each relevant to different respondents.

Interpretation

The strongest product signal is the need for skill discovery and understandable profiles. However, positive interest is not the same as willingness to exchange: only two respondents chose reciprocal participation. Two wanted help only, and two were not ready to participate. A dedicated platform may improve discovery but cannot assume a dense two-sided market.

The sample also shows that competence, teaching readiness, availability, and practical arrangements need to be visible or tested. Manual search must remain available when no reciprocal match exists. Matching explanations should show both directions of the exchange so users can judge relevance themselves.

Product implications

- Prioritize canonical offered/wanted skill profiles and skill search.
- Explain the exact pair of skills behind each reciprocal suggestion.
- Support empty and no-match states with manual-search recovery.
- Keep meeting format flexible and record arrangements outside the learning session itself.
- Treat ratings and identity verification as research questions rather than established MVP requirements.
- Validate willingness to teach, competence signals, scheduling needs, and acceptable profile fields with a larger and more representative sample.

Limitations

The questionnaire is small, self-reported, and non-representative. One response contained an unresolved inconsistency between reporting no recent need and also reporting a successful search. The study did not observe real exchanges, measure matching success, or test learning improvement. The evidence supports prototyping and further validation, not a claim of proven demand.

3. Literature review

Source document: docs/research/literature-review.md

Review scope

The targeted narrative review considers three areas: peer learning in higher education, reciprocal people-to-people recommendation, and motivation/trust/transparency. It combines foundational publications with recent studies and uses DOI or official publisher records where available. It is not a systematic review and does not claim exhaustive coverage.

Peer learning evidence

Peer learning is not one uniform intervention. Topping's taxonomy distinguishes arrangements by role continuity, group size, participant level, place, time, and objective. Structured peer tutoring can support performance and transferable skills, but context and design matter. Topping et al.'s paired calculus tutoring study reported positive outcomes but compared different cohorts, so causal conclusions remain limited.

Boud, Cohen, and Sampson show that peer learning and assessment can support feedback and shared responsibility, while grading structures can also undermine collaboration. A 2023 meta-analysis by Bengesai, Amusa, and Dhunpath found a moderately strong positive relationship between formal peer learning and undergraduate course performance across 37 studies, while noting geographical and subject concentration. Recent online and hybrid studies report positive perceptions and relationships among motivation, learning ability, and interpersonal factors, but rely on specific settings and often on self-report.

For this project, the evidence justifies studying structured peer exchange. It does not justify promising improved grades or learning outcomes before implementation and evaluation.

Reciprocal matching

Reciprocal recommendation must satisfy preferences in both directions. Unlike recommending an item, a people-to-people system can fail when one participant benefits but the other does not. The MVP therefore uses an explainable rule: suggest B to A only when A offers at least one skill B wants and B offers at least one skill A wants.

This rule is understandable and testable, but it can produce sparse results. The system should preserve manual desired-skill search and show why no match is available. Machine-learning ranking is unnecessary until the team has adequate usage data and a validated need for more complex recommendation.

Motivation, trust, and transparency

Participation depends on more than technical matching. Users need confidence that a partner is relevant, available, and safe to contact. Profiles, visible offered/wanted skills, clear request controls, and explanations give users agency. Reputation features may help but can create bias, cold-start problems, and pressure; they should be added only after validation and moderation planning.

The platform should avoid treating peer exchanges as formal credentials. Early trust measures should focus on transparent profiles, controlled disclosure, authorization, clear states, and the ability to decline or cancel.

Research gap and design decisions

Existing research supports peer-learning investigation but offers limited direct evidence about a lightweight reciprocal skill-exchange platform for university students. The project contribution is the combination of student profiles, two-sided skill matching, transparent explanations, and a simple agreement workflow.

Evidence-informed decisions are:

- retain a narrow one-to-one MVP;
- make the matching rule visible and keep manual search available;
- separate arranging an exchange from delivering or grading learning;
- evaluate discovery and usability before making claims about learning outcomes;
- collect contextual evidence from target students and preserve negative findings.

Key references

The complete bibliography is retained in `docs/research/literature-review.md`. Core sources include Topping (1996); Topping et al. (1996); Boud, Cohen, and Sampson (1999); Bengesai, Amusa, and Dhunpath (2023); Tang, Lau, and Chau (2022); and Mendieta-Aragon et al. (2023), together with research on reciprocal recommendation, motivation, trust, reputation, and explainable interfaces.

4. Existing systems comparison

Source document: docs/research/existing-systems-comparison.md

Method and selected systems

The comparison examines Simbi, Tandem, Knack, and ADPList. They represent different but relevant models: direct skill exchange, language exchange, institutional peer support, and volunteer mentoring. The review uses consistent criteria and relies primarily on official product information. Features that could not be verified are marked as unknown in the full source document.

Comparison summary

Criterion	Simbi	Tandem	Knack	ADPList
Main model	Community skill exchange	Language partner discovery	Campus tutoring/peer support	Volunteer mentorship
Profiles and discovery	Service/skill listings	Language and interest profiles	Course/campus matching	Mentor expertise and availability
Reciprocal exchange	Community-credit model rather than strict two-way pair	Mutual language practice is common	Not central	Not central
Request/agreement flow	Listings and messaging	Partner contact and communication	Institution-specific scheduling/support	Booking and mentoring sessions
Cost model	Internal community currency	Free/paid tiers	Institution dependent	Free mentoring model
Campus focus	General community	Global language community	Strong university context	Global professional community
Main lesson	Exchange needs a manageable value model	Profiles and trust matter	Campus context can improve relevance	Clear expertise and scheduling reduce uncertainty

What existing systems solve

The systems show that discovery, trust information, communication expectations, and scheduling are recurring needs. They also show that a platform does not need to provide the learning session itself: users can arrange a meeting and use external tools. University context can narrow the community and increase relevance, while global systems offer broader supply but weaker local connection.

None of the reviewed products establishes the exact proposed combination as a validated solution for AITU students. Simbi's internal currency is more complex than the MVP; Tandem is focused on languages; Knack depends on institutional structures; ADPList is mentorship rather than reciprocal exchange.

Implications for the MVP

- Use concise profiles with offered and wanted skills.
- Support direct search plus explainable reciprocal suggestions.
- Keep requests explicit: sender, recipient, skill pair, status, and history.

- Record topic, time, and in-person/external-online format after acceptance.
- Do not introduce payments, credits, built-in calls, or formal certification.
- Treat filters, ratings, and additional trust measures as validation items.

The differentiating hypothesis is that a transparent two-direction match can reduce discovery effort for students with complementary needs. This must be evaluated against ordinary one-direction skill search using the same data and task conditions.

5. Technology comparison and selected stack

Source document: docs/architecture/technology-comparison.md

Decision criteria

Options were compared for functional fit, maintainability, security responsibilities, deployment, team fit, and MVP effort. The project needs one web client, an HTTP API, relational user/skill/request data, clear ownership between two developers, and a reviewable contract.

Selected technologies

Layer	Compared options	Decision	Main reason
Frontend	React, Next.js, Vue	React + TypeScript + Vite	Component UI, static deployment, type checking, no unnecessary server-rendering layer
Backend	Node/Express, Django, Spring Boot	Node.js + Express + TypeScript	Small HTTP surface, one language across stack, explicit modular structure
Database	PostgreSQL, MongoDB, SQLite	PostgreSQL	Foreign keys, uniqueness, checks, transactions, and indexes for connected domain data
Communication	REST, GraphQL, WebSocket	REST-style JSON over HTTPS	Predictable resource/state operations with broad tooling
Contract	Informal examples vs formal schema	OpenAPI	Shared, reviewable boundary for parallel frontend/backend work

Trade-offs

React requires separate decisions for routing, forms, API state, and testing. Express provides few conventions, so the team must enforce route, validation, controller, service, repository, and domain boundaries. PostgreSQL adds migration, credential, backup, and deployment work. REST can require several requests and needs disciplined versioning and error design.

These costs are acceptable for the MVP because the selected stack avoids full-stack rendering complexity, multi-language context switching, and a flexible document model that does not match the relational domain.

Security and maintenance baseline

- Use supported dependencies and automated dependency checks.
- Require HTTPS outside local development.
- Hash passwords with a maintained adaptive password-hashing library.
- Validate input server-side and centralize safe error handling.

- Enforce authentication and object-level authorization on every protected operation.
- Use parameterized database access and least-privilege credentials.
- Restrict CORS to the deployed frontend origin.
- Protect authentication endpoints with rate limiting.
- Maintain migrations, backup/restore instructions, tests, and a synchronized OpenAPI document.

Review triggers

Reconsider the stack if university constraints mandate another platform, a setup exercise reveals a serious skill gap, validated requirements add real-time collaboration or multiple clients, the data becomes predominantly unstructured, or hosting constraints make separate frontend/backend/database deployment impractical.

6. Requirements

Source document: docs/requirements.md

Scope and prioritization

The initial requirements cover the complete journey from registration to an agreed one-to-one exchange. Priorities use MoSCoW. Must requirements are required for the main scenario; Should requirements improve quality but are not mandatory for the demonstration; Could requirements are optional. Assumptions about credentials, exact profile fields, filters, and target values require validation.

Functional requirements summary

Area	Mandatory behaviour
Authentication	Register with unique email, sign in without account disclosure, sign out and invalidate the session
Profiles	View/edit own profile; view approved public fields of another authenticated student's profile
Skills	Add/remove offered and wanted skills from a shared catalogue; prevent duplicates
Search	Find other users by offered skill/category; exclude self; provide recoverable empty states
Matching	Return only reciprocal pairs; show skills in both directions; distinguish incomplete-profile and no-match states
Requests	Send valid requests; prevent self/invalid/duplicate pending requests; show incoming/outgoing lists
State changes	Recipient accepts/declines pending request; sender cancels pending request; terminal transitions are rejected
History	Record immutable status, actor, and timestamp for creation and transitions
Agreement	For an accepted request, participants record/edit topic, future time, and in-person or external-online format

State model

Initial state	Actor/action	Result
No request	User sends a valid request	`Pending` plus initial history entry
Pending	Recipient accepts	`Accepted` plus history entry
Pending	Recipient declines	`Declined` plus history entry
Pending	Sender cancels	`Cancelled` plus history entry
Pending	Wrong actor changes status	Rejected; no data change
Accepted	Participant edits agreement	Status remains `Accepted`; agreement and audit fields update
Accepted/Declined/Cancelled	Any terminal transition	Rejected; no data change

Matching rule

For current user A, user B is a reciprocal suggestion when A offers at least one active skill wanted by B and B offers at least one active skill wanted by A. A is excluded from their own results. Explanations show at least one current skill in each direction. If either profile lacks offered or wanted skills, the interface prompts completion. If the profile is complete but no pair exists, manual search remains available.

Non-functional requirements

Category	Required baseline
Security/privacy	Adaptive password hashes; authenticated protected endpoints; object-level authorization; validation; HTTPS; safe errors; no secret/private fields in responses
Usability	Main scenario understandable without moderator guidance; visible loading/success/error/empty states; clear field errors
Accessibility	Keyboard-operable core journey; visible focus; labels; tested components meet WCAG 2.2 AA contrast
Performance	Under the defined test load, p95 <=500 ms for profile/request operations and <=1,000 ms for search/matching; API error rate below 1%
Compatibility	Latest two stable major versions available at test time of Chrome, Firefox, Edge, and Safari
Responsive design	Core pages at 360, 768, 1024, and 1440 px without unintended horizontal scrolling or hidden primary actions
Reliability	Status update and history insertion are atomic; failed operations do not leave partial data
Maintainability	Versioned OpenAPI contract, modular code, migrations, automated checks, and traceability from every Must requirement to evidence

Traceability and validation

Each requirement retains its identifier, source, priority, acceptance criteria, and planned test IDs in the full requirements and evaluation plan. Student research supports prioritizing skill search and offered/wanted profiles but does not validate all profile fields or prove reciprocal participation. Ratings, verification, built-in chat, notifications, and advanced filters remain potential later work unless further evidence changes the MVP.

7. System architecture

Source document: docs/architecture/system-architecture.md

Architecture style and drivers

The proposed system uses a browser single-page application, one modular backend service, and PostgreSQL. It is intentionally a modular monolith for the MVP: one deployable backend keeps transactions and authorization understandable while internal modules preserve separation. The main drivers are data integrity, object-level access control, reciprocal matching, consistent request states, independent frontend/backend work, and an API that can be tested directly.

Components and responsibilities

Component	Responsibilities
React frontend	Routes, forms, profile/skill management, search, match explanations, request lists, agreements, loading/empty/error states
Express API	HTTPS/JSON endpoints under `/api/v1`, authentication boundary, input validation, controllers, safe errors
Domain services	Authorization, matching, request transitions, agreement rules, transaction coordination
Repositories	Parameterized persistence operations without business-policy decisions
PostgreSQL	Users, skills, offered/wanted relations, requests, immutable status history, agreements, constraints, indexes
OpenAPI contract	Shared schemas, operations, authentication, responses, and error codes for client/server integration

Principal data relationships

A user has many offered skills and many wanted skills through unique join records that point to a canonical skill catalogue. An exchange request links sender, recipient, offered skill, wanted skill, current status, and creation time. Each status change appends a history record with actor and timestamp. An agreement belongs to an accepted request and stores topic, future meeting time, format, and format-specific details.

Runtime flows

Reciprocal matching: the client requests suggestions; the service authenticates the user, loads active offered/wanted relationships, finds users satisfying both overlap directions, excludes self, and returns explanations. Missing data and zero matches produce distinct responses.

Exchange request: the API validates identities, active skills, duplicate status, and authorization. Creation of `Pending` plus the initial history record is atomic. Recipient accept/decline and sender cancellation each validate the current state and write the new state/history atomically.

Agreement: only participants of an accepted request can create or update the shared plan. Topic, future time, and format-specific fields are validated. The request remains accepted; audit fields identify the last update.

Security and deployment

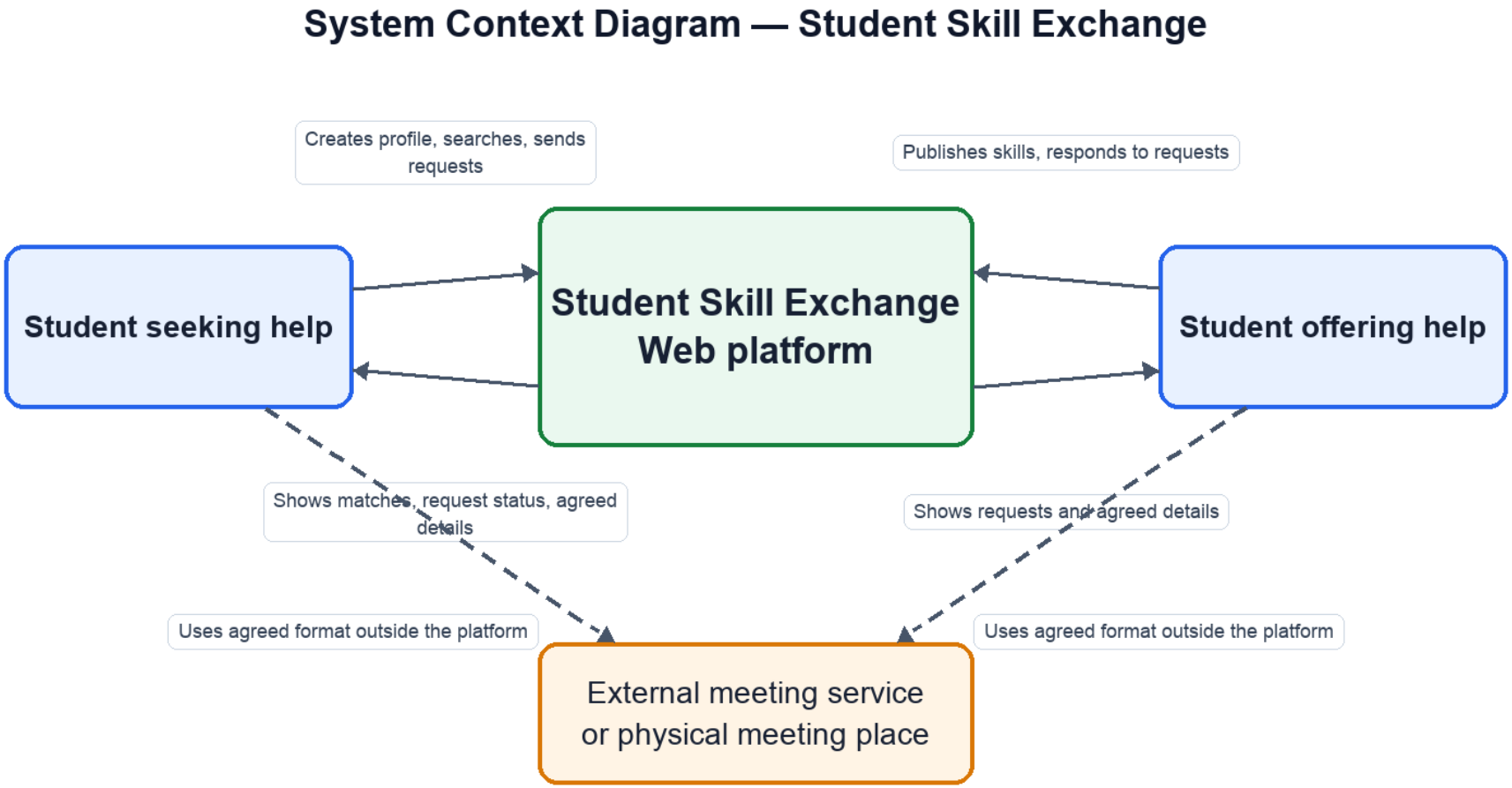
The frontend is not a security boundary. The API authenticates each protected operation, applies object-level authorization, validates input, and exposes only approved fields. Passwords are hashed; secrets remain outside the repository; HTTPS and restricted CORS are used in deployment. The database uses constraints, transactions, migrations, least-privilege credentials, and backups.

Initial deployment consists of static frontend assets, one Node/Express service, and one PostgreSQL database. Provider selection remains open because university constraints, cost, and account availability are not confirmed.

Open decisions

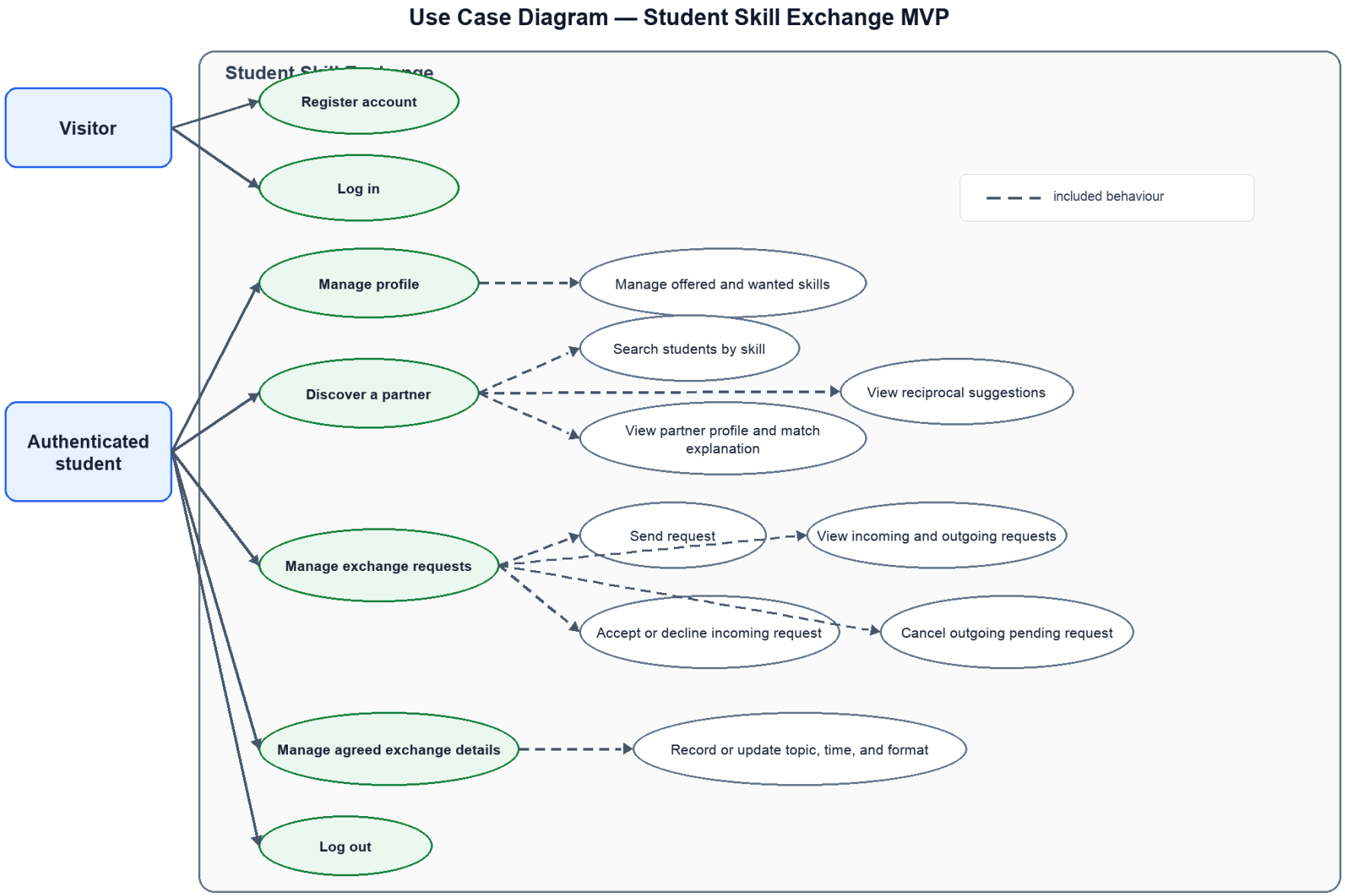
Session/token design, final profile visibility, hosting provider, email delivery, skill-catalogue governance, moderation, and any later reputation mechanism require separate reviewed decisions. The architecture does not claim that these unresolved items are implemented.

System context diagram



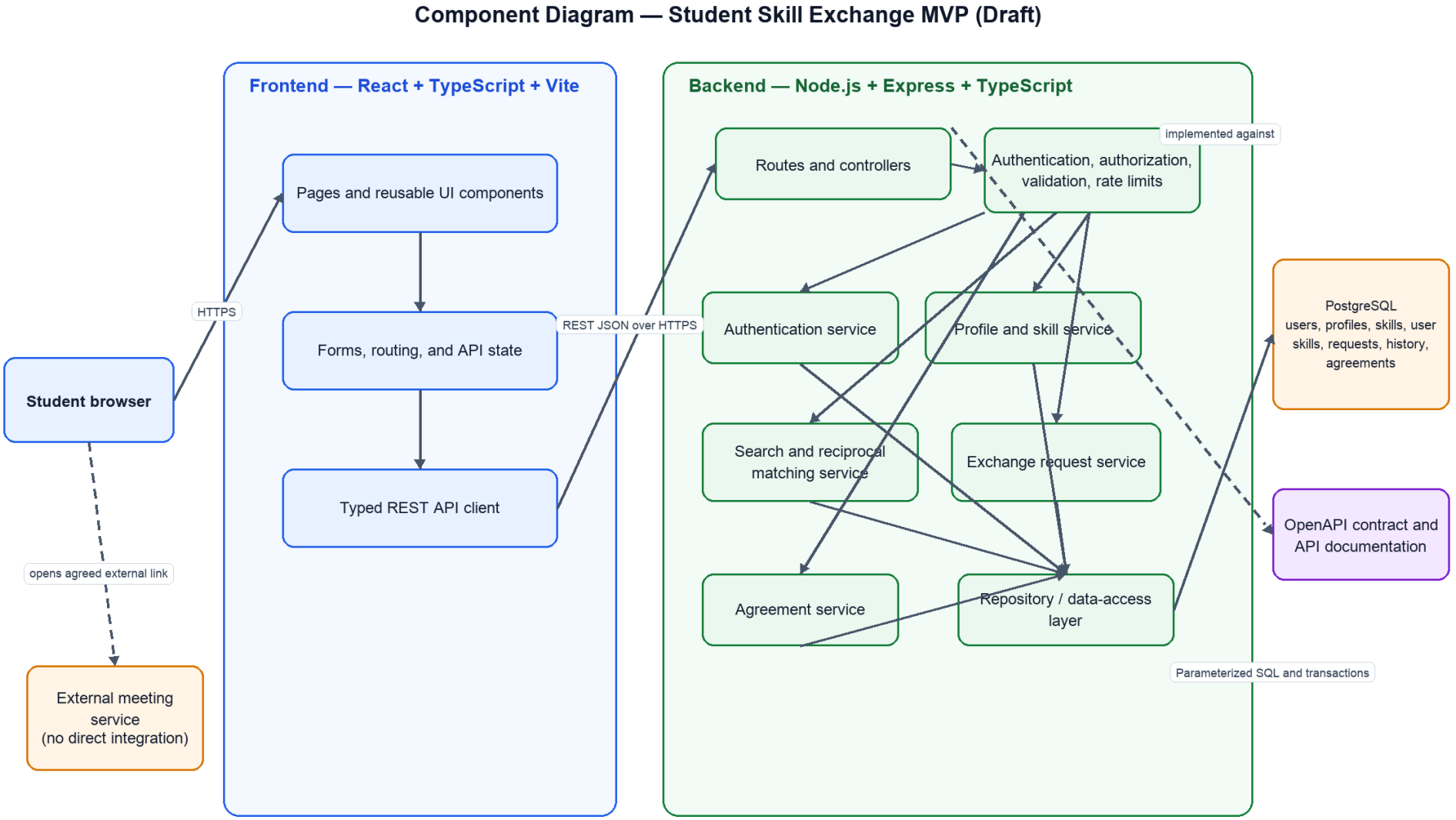
System context diagram

Use-case diagram



Use-case diagram

Component diagram



Component diagram

8. Interactive UI prototype

Source artifact: SSE-8_UI_Prototype.zip

Purpose

The repository contains a self-contained interactive HTML prototype for the main Student Skill Exchange journey. It demonstrates the intended interaction model before implementation and uses sample data in the browser.

Demonstrated behaviour

Prototype area	Demonstrated behaviour
Account/profile	Sample registration/sign-in, personal details, offered skills, and wanted skills
Discovery	Search and category filtering across sample profiles
Reciprocal matching	Both exchange directions and the skills that explain each suggestion
Requests	Create, accept, decline, cancel, and inspect incoming/outgoing states
Shared plan	Record topic, Astana date/time, and in-person or external-online format
UI states	Normal, loading, validation, empty, recoverable error, duplicate request, and authorization feedback
Accessibility/responsiveness	Visible focus, labels, error messages, and layouts for common screen widths

Evaluation role and limitation

The prototype supports early walkthroughs and usability refinement. It is not evidence of a deployed system. Data persistence, real authentication, backend APIs, database constraints, multi-user concurrency, security controls, production performance, and completed verification remain implementation work. Findings from prototype interaction must be labelled separately from results obtained on the future connected application.

9. Verification and evaluation plan

Source document: docs/testing/verification-and-evaluation-plan.md

Evidence policy

All tests in this section are planned. No implementation test result is claimed. At execution time, every run must record build/commit, environment, data version, browser/API tool, date, result, and evidence link. Failed, blocked, and skipped cases remain visible.

Verification strategy

Level	Main coverage
Static/unit	Validation, matching rule, state guards, format rules, and pure transformations
API/integration	Authentication, object authorization, database constraints, transactions, history, errors, and OpenAPI conformance
UI/end-to-end	Successful main journey plus invalid input, loading, empty, permission, and recoverable-error states
Non-functional	Security, accessibility, performance, compatibility, responsiveness, reliability, and maintainability
User evaluation	Unaided task completion, discovery comparison, comprehension, effort, and feedback

Core functional catalogue

Area	Planned checks
Authentication	Unique registration, invalid fields, non-disclosing login errors, sign-out/session invalidation
Profiles/skills	Persistence, validation, cross-user denial, duplicate prevention, catalogue consistency, historical preservation
Search/matching	Filters, self-exclusion, empty recovery, reciprocal rule, explanations, incomplete-profile/no-match distinction
Requests	Valid creation, self/invalid/duplicate rejection, participant-only lists, actor/state permissions, immutable history
Agreement	Accepted-state requirement, participant authorization, future time, format fields, update audit, no built-in chat/video

Non-functional targets

Area	Planned target
Authorization	Anonymous `401`; unauthorized object access `403` or non-disclosing `404`; permitted access succeeds
Validation	Invalid and injection-like input is treated safely; no unrelated data changes
Credential/security	Adaptive salted hashes, secure cookie/token design, HTTPS, no committed active secret
Accessibility	Keyboard completion, visible focus/labels, no critical automated violation, WCAG 2.2 AA contrast for tested components

Area	Planned target
Performance	20 concurrent users; 1,000 users/10,000 skill relations; p95 <=500 ms common operations and <=1,000 ms search/matching; <1% API errors
Compatibility	Main scenario on latest two stable majors available at test time of Chrome, Firefox, Edge, and Safari
Responsiveness	Core screens at 360/768/1024/1440 px without clipping or hidden primary action
Reliability	Induced failure between state/history writes rolls back both; successful transaction commits both
Contract	Versioned OpenAPI validates and matches implemented operations, schemas, auth, responses, and errors

Evaluation design

Five representative university students who did not implement the tested screens will complete the main scenario on a staging build. Participation is voluntary, data is anonymized, and unnecessary personal information is not collected. A scenario succeeds only when all required records are stored and the participant completes without moderator navigation assistance.

The matching condition is compared with a baseline of manual desired-skill search. Participants complete equivalent tasks with counterbalanced order and equally sized synthetic datasets. The matching condition shows reciprocal suggestions and explanations; the baseline requires inspecting ordinary search results.

Metric	Target
Full main-scenario completion	At least 4 of 5 without assistance
Correct partner selection in matching condition	At least 4 of 5
Explanation comprehension	At least 4 of 5 correctly state both skill directions
Discovery time	Matching median no longer than baseline; 20% reduction is desired, not a release gate
Discovery effort	Matching median profile/result views no higher than baseline
Critical errors	Zero unresolved critical errors at release
Task ease	Median at least 5 on a 1-7 scale
Perceived usefulness	Median at least 4 on a 1-5 scale

Entry, suspension, and exit

Testing starts only after requirements/architecture are merged, the feature is reviewed, automated checks pass, staging and test data are ready, OpenAPI matches, and participants receive consent information. Testing pauses for data-integrity loss, a broken main journey, possible participant-data exposure, or any blocker that invalidates later results.

Release requires evidence for every Must requirement, no unresolved critical/high defect affecting the main journey or protection, mandatory non-functional targets met or a formally approved corrective plan, 4/5 unaided completion, and a final report that separates measurements from interpretation.

10. Project management, responsibilities, and risks

Source document: docs/planning/project-plan-and-risks.md

Delivery model

Each work item has one assignee and the other member as peer reviewer. Nursultan leads backend/data, technical evidence, API/security, and assessment assembly. Tamir leads frontend/UI, user evidence, planning coordination, usability, and user documentation. Both agree the contract, integrate increments, test the main journey, maintain risks, and preserve contribution evidence.

Milestones

Milestone	Proposed window	Exit condition
M1 Research/requirements refinement	Week 1	Method limits recorded, findings reflected in priorities, assumptions have owners/actions
M2 Design/API agreement	Week 2	Data model, diagrams, flows, state rules, OpenAPI examples/errors/timezone handling reviewed
M3 First connected implementation	Week 3	Registration and profile editing work through frontend, API, and database; persistence survives reload
M4 Core features	Weeks 4-5	Skills, search, matching, requests/history, and agreements delivered with negative checks
M5 Integration candidate	Week 6	Full two-user journey works; persistence, contract, migrations, and setup are reproducible
M6 Testing/evaluation	Week 7	Must-requirement evidence, security/state checks, compatibility, usability, and baseline comparison recorded
M7 Final documentation/demo	Week 8	Report, diagrams, API/database/setup/test/user guides, contribution evidence, and demo match tested version

These windows are proposed and begin after the checkpoint; they are not official academic dates. Calendar targets must be confirmed against capacity and university deadlines.

Key dependencies

Research and technology decisions refine requirements; requirements drive architecture, prototype, and verification. The data model and API contract enable parallel frontend/backend work. Identity precedes ownership, skills precede reciprocal matching, and an accepted request precedes the shared agreement. Evaluation requires a connected and reproducible candidate; final claims require measured results.

Priority risks

Risk	Likelihood / impact	Main response	Owner
Deadline or review evidence missing	High / High	Inventory mandatory evidence, focus PRs, retain original deadlines and document remaining work	Tamir
Unconfirmed academic dates conflict with plan	Medium / High	Confirm official cutoffs and re-plan against actual capacity	Tamir
Small survey is rejected or overstated	Medium / High	Preserve method/limits, seek allowed follow-up evidence, avoid representative claims	Tamir
Reciprocal matching yields sparse results	Medium / High	Measure coverage, retain manual search, explain no-match states	Tamir
API/client contract drift	Medium / High	Version/review OpenAPI, use fixtures and contract checks, integrate early	Nursultan
Unauthorized object access	Medium / High	Server-side authorization matrix and cross-user negative tests; block release on exposure	Nursultan
Concurrent actions corrupt state/history	Medium / High	Guard transitions, use transactions, test duplicates/concurrency/failure rollback	Nursultan
Accessibility/responsive blockers	Medium / High	Keyboard, state, browser, and width matrix checks; fix main-flow blockers	Tamir
Evaluation lacks valid conditions	Medium / High	Freeze build/data/protocol, arrange participants, record failures and gaps	Tamir
Account, link, access, or tag traceability fails	Medium / High	Identify accounts, retain PR/task evidence, check permissions and package consistency	Nursultan

Workflow and definition of done

Work starts from `main`, uses a task branch, coherent commits, a linked pull request, peer review, applicable checks, merge, and permanent evidence in YouTrack. Documentation tasks require content and consistency review. Implementation additionally requires relevant unit, contract, integration, permission, and negative-case checks plus updated setup documentation.

Progress is measured by accepted outputs, Must requirements with evidence, unresolved core-flow defects, and dependency/risk actions, not by commit count alone.

11. Supervisor feedback and revisions

Source document: docs/supervision/supervisor-feedback.md

Review record

Field	Record
Review date	23 October 2026
Format	Verbal feedback
Participants	Konstantin Latuta and Nursultan Makhmutov
Evidence basis	Nursultan's paraphrased record; no recording or verbatim transcript

The supervisor reviewed the project topic, purpose, intended scope, and the plan to prepare the required materials. No detailed document-by-document or line-level comments were provided.

Feedback and response

ID	Supervisor feedback	Team response	Owner/status
SF-01	The project was approved to continue.	Continue with the approved topic, aim, and current scope.	Both members; no change required
SF-02	The team should prepare and write the project materials independently.	Author documentation in the repository, preserve sources/decisions, and use peer review.	Nursultan with Tamir as reviewer; in progress

No correction or content revision was requested. The feedback record itself is the documentation change: it preserves the approval, instruction, response, owner, and status without inventing comments.

Outcome and next steps

- Overall outcome: approved to continue.
- Required revisions: none specified.
- Team responsibility: students prepare the written materials independently.
- Further review: no date specified; ask the supervisor when a concrete academic, scope, or evidence question arises.
- Continue implementation and evaluation through reviewed backlog items after the pre-defense checkpoint.

Submission record

The assessment materials were reviewed in GitHub pull request #16 and merged into main. The reviewed commit was tagged as pre-defense-1. A separate PDF contains readable YouTrack board and task evidence. The university form and its confirmation are administrative evidence and should be stored or linked without modifying the fixed submitted tag.

Evidence	Link
Repository	https://github.com/HashiraOfSun/student-skill-exchange
Assessment PR	https://github.com/HashiraOfSun/student-skill-exchange/pull/16
Release/tag	https://github.com/HashiraOfSun/student-skill-exchange/releases/tag/pre-defense-1
Submission files	https://github.com/HashiraOfSun/student-skill-exchange/tree/main/docs/submission
YouTrack board	https://student-skill-exchange.youtrack.cloud/agiles/204-1/current
SSE-12	https://student-skill-exchange.youtrack.cloud/issue/SSE-12